2026 Spring – UAV Control Special Topics : Homework 2

Intelligent Vehicle & Mechatronics lab Advisor : Prof. Kang Li Author : Chih-Feng Huang

1. ROS2 Beginner

https://docs.ros.org/en/foxy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Cpp-Publisher-And-Subscriber.html

Understanding the internal behavior of a Node is essential since it is one of the core components in ROS 2. In this assignment, we will use the tutorial example publisher_member_function.cpp as the reference code. Your task is to **analyze the program** and answer the following questions to demonstrate your understanding of C++ and ROS 2 Node mechanisms.

1.1 In C++, #include can use either <> or "". What is the difference between these two forms?

1.2 Compared with traditional time-handling functions such as time_t, clock(), and sleep(), what advantages does the <chrono> library provide?

1.3 The <memory> header introduces **smart pointers**. What advantages do smart pointers have compared to traditional pointers? What are the three main types of smart pointers in C++? Briefly describe how each of them is used.

1.4 In the function main(int argc, char* argv[]), what do argc and argv represent? Why does rclcpp::init(argc, argv); also require these arguments?

1.5 In this program, class MinimalPublisher inherits from rclcpp::Node.

- What is **inheritance** in C++?
- What are the advantages of using inheritance?
- In this example, public inheritance is used. What other inheritance types exist, and how do they differ?
- Why is public inheritance appropriate in this case?
- The functions create_publisher() and create_wall_timer() belong to the Node class. Why can they be called using this-> inside MinimalPublisher?

1.6 Inside the public section of class MinimalPublisher, a function named MinimalPublisher() is defined. This is the constructor of the class.

- What is a constructor in C++?
- Why does it use the syntax : Node("minimal_publisher") after the constructor declaration?

1.7 The statement timer_ = this->create_wall_timer() uses std::bind(), which comes from #include <functional>. Why is std::bind() needed in this context?

1.8 Callbacks are an important concept in ROS 2.

- What is a callback?
- Another approach is called polling. What is the difference between callback and polling, and why is callback generally more efficient?

1.9 What role does rclcpp::spin() play in ROS 2? How are callbacks executed? Explain the relationship between **Node**, **Executor**, and **Callback**.

1.10 Why does a ROS 2 publisher require **templates**?

1.11 When the program runs, describe the overall execution flow of the system, including the following stages (main()、rclcpp::init、Node creation、Timer creation、Executor spin、Callback execution)

2. 3D Coordinate Transformation and Rigid Body Motion (C++ Implementation)

General Instructions

- Implement all problems **using C++ with the Eigen library**
- All quaternions must be **normalized before use**
- Output all numerical results with **6 decimal precision**
- **Do NOT hardcode** results
- Clearly structure your code for readability

2.1 Multi-Point Coordinate Transformation

Given two coordinate frames R_1 and R_2, both defined in the world frame W:

$$q_1 = [0.7, 0.1, 0.2, 0.6]^T, t_1 = [0.2, 0.3, -0.1]^T$$

$$q_2 = [0.9, -0.1, 0.2, 0.3]^T, t_2 = [-0.4, 0.1, 0.5]^T$$

These represent transformations T_{R_1W} and T_{R_2W}. Five points are given in frame R_1:

$$p_1 = [0.1, 0, 0]^T, p_2 = [0, 0.2, 0]^T, p_3 = [0, 0, 0.3]^T, p_4 = [0.2, 0.2, 0.2]^T, p_5 = [-0.1, 0.1, 0.4]^T$$

Tasks:

i. Transform all points from R_1 to R_2

ii. Output all transformed points p_{R_2}

iii. Compute the centroid of the transformed points

2.2 Reverse Transformation (Validation)

Using the results from 2.1:

Tasks:

i. Transform all points from R_2 back to R_1

ii. Obtain reconstructed points p'_{R_2}

iii. Compute the error for each point: $e_i = || p'_{R_1,i} - p'_{R_1,i} ||$

iv. Output: Error for each point、Maximum error

Note

Incorrect transformation direction will result in large errors.

2.3 **Relative Pose Computation**

Given two frames A and B defined in the world frame W:

$$q_A = [0.75, 0.25, 0.1, 0.6]^T, t_A = [0.4, 0.2, 0.1]^T$$

$$q_B = [0.65, -0.15, 0.45, 0.65]^T, t_B = [0.2, 0.5, 0.3]^T$$

These represent transformations T_{AW} and T_{BW}.

Tasks

i. Compute the relative transformation: $T_{BA} = T_{BW} T_{AW}^{-1}$

ii. Output : Rotation Matrix R_{BA} and Tranlation vector t_{BA}

2.4 **World to Camera Transformation**

Given two camera frames C_1 and C_2 :

$$q_1 = [0.85, 0.25, -0.2, 0.3]^T, t_1 = [0.3, 0.2, 0.4]^T$$

$$q_2 = [0.7, -0.3, 0.1, 0.5]^T, t_2 = [-0.5, 0.1, 0.2]^T$$

A point in the world frame is:

$$p_W = [1.2, -0.4, 0.8]^T$$

Tasks

i. Compute p_{c_1} and p_{c_2}

ii. Compute their distances to the origin

iii. Determine which camera is closer to the point

2.5 **Effect of Quaternion Normalization**

Given non-normalized quaternions:

$$q_1 = [2.0, 0.5, 1.0, 1.5]^T, t_1 = [0.2, 0.1, 0.3]^T$$

$$q_2 = [1.5, -0.4, 0.2, 1.0]^T, t_2 = [-0.1, 0.4, 0.2]^T$$

$$p_{R_1} = [0.4, 0.2, -0.1]^T$$

Tasks

i. Compute transformation without normalization

ii. Compute transformation with normalization

iii. Compare the results

iv. Provide a brief explanation (≤100 words)

2.6 **Manual SE(3) Construction**

Given:

$$q = [0.9,0.1,0.2,0.3]^T, t = [0.5, -0.2,0.1]^T$$
$$p = [0.2,0.4, -0.1]^T$$

Tasks

i. Convert quaternion to rotation matrix

ii. Convert quaternion to rotation matrix

iii. Convert point to homogeneous coordinates

iv. Compute transformed point

Requirement

Do NOT use Eigen built-in transformation classes (e.g., Isometry3d)

2.7 **Chain Transformation Across Multiple Frames**

Given three frames R_1 , R_2 , R_3 defined in world frame W :

$$q_1 = [0.8 , 0.1, \ 0.2, \ 0.5]^T, t_1 = [0.3, 0.1, -0.2]^T$$
$$q_2 = [0.6, -0.2, 0.4, 0.5]^T, t_2 = [-0.4, 0.2, 0.3]^T$$
$$q_3 = [0.7, 0.3, -0.1, 0.6]^T, t_3 = [0.1, -0.5, 0.4]^T$$

These represent the transformations T_{R_1W}, T_{R_2W} and T_{R_3W}, respectively.

A point is given in frame R_1 :

$$p_{R_1} = [0.2, -0.1, 0.6]^T$$

Tasks

i. Compute p_{R_2} and p_{R_3}

ii. Compute p_{R_3} using the following two methods:

- Method 1：$R_1 \rightarrow R_2 \rightarrow R_3$
- Method 2：$R_1 \rightarrow W \rightarrow R_3$

iii. Compare the two results and compute the error between them.

Requirements

i. Normalize all quaternions before use

ii. Clearly show the transformation chain used in each method

iii. Output all results with 6 decimal precision